RELATÓRIO TÉCNICO

Consolidação da ETL e Preparação da Infraestrutura de Execução

AdventureWorks Data Warehouse

Disciplina: Projeto de Análise e Fluxo de Dados - OLAP e ETL
Projeto individual - Demonstração de Competência
Data do registro: 23 de outubro de 2026

Situação ao final da sessão: Código ETL consolidado; SQL Server e PostgreSQL DW validados; modelo físico e controle incremental criados. Primeira extração real da dimensão produto ainda pendente para a próxima sessão.

1. Objetivo e escopo desta etapa

Objetivo. Consolidar a lógica ETL reutilizada do projeto anterior, adequá-la ao novo modelo dimensional oficial, preparar as conexões com as bases de origem e destino e validar a infraestrutura necessária para executar a carga incremental posteriormente com Apache Airflow.

Escopo efetivamente realizado. Foram revisados e adaptados os módulos de extração, transformação, carga, controle de watermark e conexão; foi configurado o acesso do container ao SQL Server AdventureWorks; foi criado um PostgreSQL exclusivo para o DW; os scripts SQL do modelo físico e do controle incremental foram executados; e as conexões foram testadas nos dois sentidos da futura ETL.

Limite desta sessão. A carga completa da ETL e a execução por DAG ainda não foram realizadas. A próxima ação é testar a extração real de dim_produto e avançar gradualmente para transformação, carga e pipeline completo.

2. Estrutura consolidada do projeto

A organização adotada separa responsabilidades e facilita a integração futura com as DAGs do Airflow:

```
etl/
├── extract/
│   ├── extrair_dim_produto.py
│   ├── extrair_dim_territorio.py
│   ├── extrair_dim_tempo.py
│   └── extrair_fato_venda.py
├── transform/
│   ├── transformar_dim_produto.py
│   ├── transformar_dim_territorio.py
│   ├── transformar_dim_tempo.py
│   └── transformar_fato_venda.py
├── load/
│   ├── carregar_dim_produto.py
│   ├── carregar_dim_territorio.py
│   ├── carregar_dim_tempo.py
│   └── carregar_fato_venda.py
├── incremental/
│   └── pipeline_incremental.py
├── control/
│   ├── watermark.py
│   └── buscar_chaves_dimensoes.py
├── tests/
└── conexoes.py
```

Scripts SQL atualmente consolidados:

```
sql/
├── 01_modelo_fisico_dw_postgresql_final.sql
├── 02_controle_etl.sql
└── 04_kpis_final.sql
```

3. Adaptação dos módulos ETL

3.1 Extração

Módulo	Função principal	Critério incremental

Módulo	Função principal	Critério incremental
extrair_dim_produto.py	Extrai produto, subcategoria, categoria, preço e ModifiedDate do SQL Server.	ModifiedDate > watermark quando informado.
extrair_dim_territorio.py	Extrai território de vendas, país, grupo territorial e ModifiedDate.	ModifiedDate > watermark quando informado.
extrair_dim_tempo.py	Obtém o intervalo de datas das vendas e gera a dimensão calendário em Python.	Não utiliza watermark; dimensão determinística.
extrair_fato_venda.py	Extrai cabeçalho/detalhe de venda, território, produto, valores, desconto e custo histórico.	Maior ModifiedDate entre cabeçalho e detalhe.

A extração da fato utiliza o custo histórico do produto quando disponível e mantém fallback para o custo atual. As funções recebem a conexão como parâmetro, eliminando credenciais fixas no código.

3.2 Transformação

- Padronização dos nomes de colunas para o modelo físico oficial do schema dw.
- Conversão e tratamento de tipos numéricos, datas, textos nulos e duplicidades.
- Uso de valores padrão como “Não informado” e “Não classificado” nas dimensões quando necessário.
- Geração determinística de sk_tempo no formato AAAAMMDD.
- Resolução de sk_produto e sk_territorio a partir das chaves de negócio; quando não encontrados, uso do membro desconhecido -1.
- Cálculo de margem_bruta = valor_receita - custo_total na fato.
- Remoção da antiga dependência de dim_cliente, pois ela não faz parte do modelo dimensional oficial desta versão.

3.3 Carga

Os módulos de carga foram alinhados às tabelas oficiais e implementam idempotência por meio de ON CONFLICT:

Destino	Estratégia
dw.dim_produto	UPSERT por product_id_oltp, comportamento SCD Tipo 1.
dw.dim_territorio	UPSERT por territory_id_oltp, comportamento SCD Tipo 1.
dw.dim_tempo	INSERT com ON CONFLICT (sk_tempo) DO NOTHING.
dw.fato_venda	UPSERT por (sales_order_id, sales_order_detail_id), preservando o grão da fato.

Cada rotina controla commit/rollback e não fecha a conexão recebida externamente, deixando o gerenciamento de conexão sob responsabilidade do orquestrador.

3.4 Controle incremental

Foi centralizada a lógica incremental no arquivo pipeline_incremental.py. A ordem planejada de execução é:

```
dim_produto → dim_territorio → dim_tempo → fato_venda
```

O watermark é consultado antes da extração e atualizado somente após carga bem-sucedida. Caso ocorra falha depois de uma carga e antes da atualização do watermark, a reexecução permanece segura porque as tabelas utilizam UPSERT/ON CONFLICT, reduzindo o risco de duplicidade.

4. Configuração e validação do SQL Server de origem

O SQL Server local foi preparado para permitir acesso a partir dos containers Docker:

- TCP/IP habilitado na instância padrão MSSQLSERVER.
- Porta TCP 1433 definida e serviço SQL Server reiniciado.
- Modo de autenticação alterado para SQL Server e Windows Authentication.
- Login dedicado etl_adventureworks criado para a ETL, com permissões de leitura apenas nas tabelas necessárias.
- Acesso do Docker ao host validado via host.docker.internal:1433.

As credenciais passaram a ser lidas por variáveis de ambiente em etl/conexoes.py. Nenhuma senha é registrada neste relatório ou deve ser versionada no Git.

Parâmetro	Valor de configuração
Driver	ODBC Driver 18 for SQL Server
Host no container	host.docker.internal
Porta	1433
Base de origem	AdventureWorks2016
Usuário	etl_adventureworks
Senha	Definida somente no .env local (não versionada)

Validação executada: o módulo etl.conexoes conectou-se ao SQL Server a partir da imagem apache/airflow:3.3.1 e retornou 504 registros em Production.Product e 121.317 registros em Sales.SalesOrderDetail.

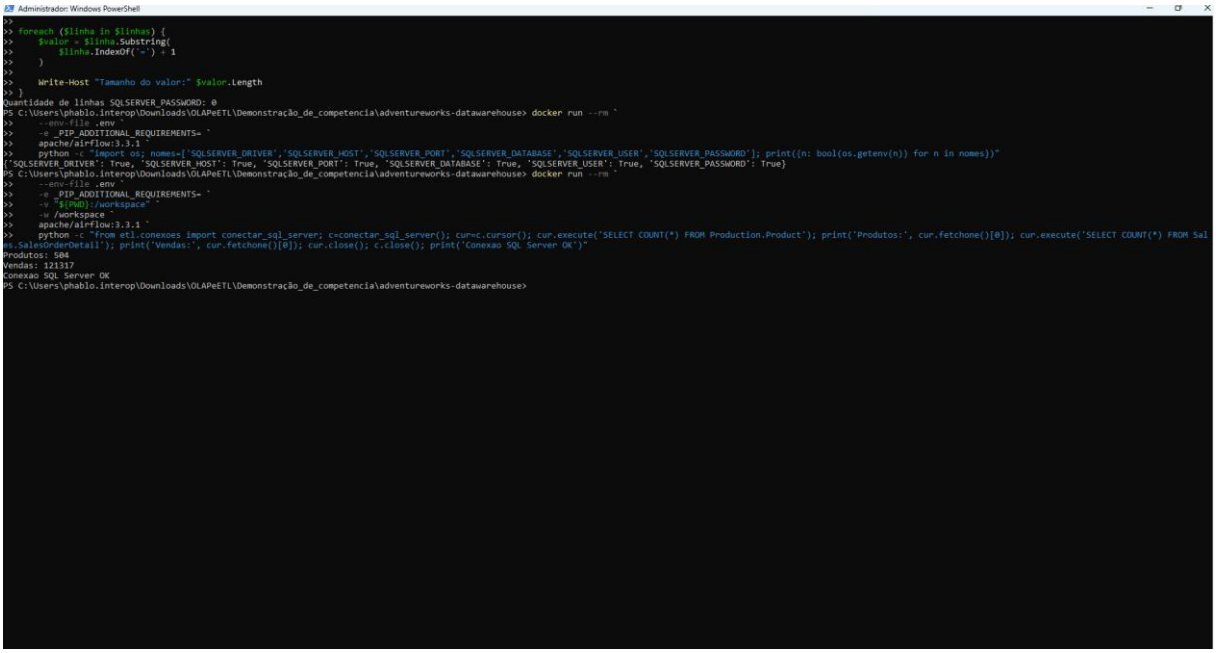


Figura 1 - Validação da conexão do container com o SQL Server AdventureWorks.

5. Infraestrutura Docker, Airflow e PostgreSQL DW

O docker-compose.yml reutilizado do material do professor foi adaptado ao projeto atual. As principais alterações foram:

- Nome do projeto Compose alterado de aula-etl para adventureworks-etl, evitando conflito com o ambiente didático original.
- Criação do serviço postgres-dw, PostgreSQL 16 Alpine exclusivo para o Data Warehouse.
- Volume persistente adventureworks-dw-volume para os dados do DW.
- Montagem das pastas ./etl e ./sql em /opt/airflow/etl e /opt/airflow/sql.
- Definição de PYTHONPATH=/opt/airflow para permitir imports como from etl....
- Separação entre o PostgreSQL de metadados do Airflow e o PostgreSQL do Data Warehouse.
- Porta de host do DW definida como 5433. A porta externa do PostgreSQL de metadados foi ajustada para 5434, pois a 5432 já estava ocupada no Windows.

Componente	Host/porta no Windows	Endereço dentro da rede Docker	Finalidade
SQL Server	localhost:1433	host.docker.internal:1433	Origem AdventureWorks2016
PostgreSQL DW	localhost:5433	postgres-dw:5432	Destino adventureworks_dw
PostgreSQL Airflow	localhost:5434	postgres:5432	Metadados do Airflow
Airflow API/UI	localhost:8080	airflow-apiserver:8080	Orquestração e interface

A estrutura do Compose foi validada com docker compose config e docker compose config --services. O serviço postgres-dw foi iniciado isoladamente e atingiu estado healthy.

6. Criação e validação do Data Warehouse

O script 01_modelo_fisico_dw_postgresql_final.sql foi executado com ON_ERROR_STOP=1 contra o banco adventureworks_dw. A execução concluiu com COMMIT, sem erros.

Tabela	Papel no modelo
dw.dim_tempo	Dimensão calendário com chave determinística AAAAMMDD.
dw.dim_produto	Dimensão de produto com chave substituta e chave de negócio product_id_oltp.
dw.dim_territorio	Dimensão territorial com chave substituta e territory_id_oltp.
dw.fato_venda	Fato no grão de item de pedido: SalesOrderID + SalesOrderDetailID.

Também foram criados os membros desconhecidos (-1) em produto e território, índices nas chaves estrangeiras da fato, restrições e comentários do modelo.

A consulta ao information_schema confirmou exatamente quatro tabelas no schema dw: dim_produto, dim_tempo, dim_territorio e fato_venda.

7. Controle de watermark incremental

O script 02_controle_etl.sql foi executado com sucesso e criou o schema controle e a tabela controle.etl_watermark. Foram inseridos os três processos que dependem de ModifiedDate:

Processo	ultima_execucao ao final da sessão
dim_produto	NULL
dim_territorio	NULL
fato_venda	NULL

O valor NULL é esperado antes da primeira carga: indica que a próxima execução será tratada como carga inicial. A dimensão tempo não possui watermark, pois é gerada deterministicamente e sua carga utiliza ON CONFLICT DO NOTHING.

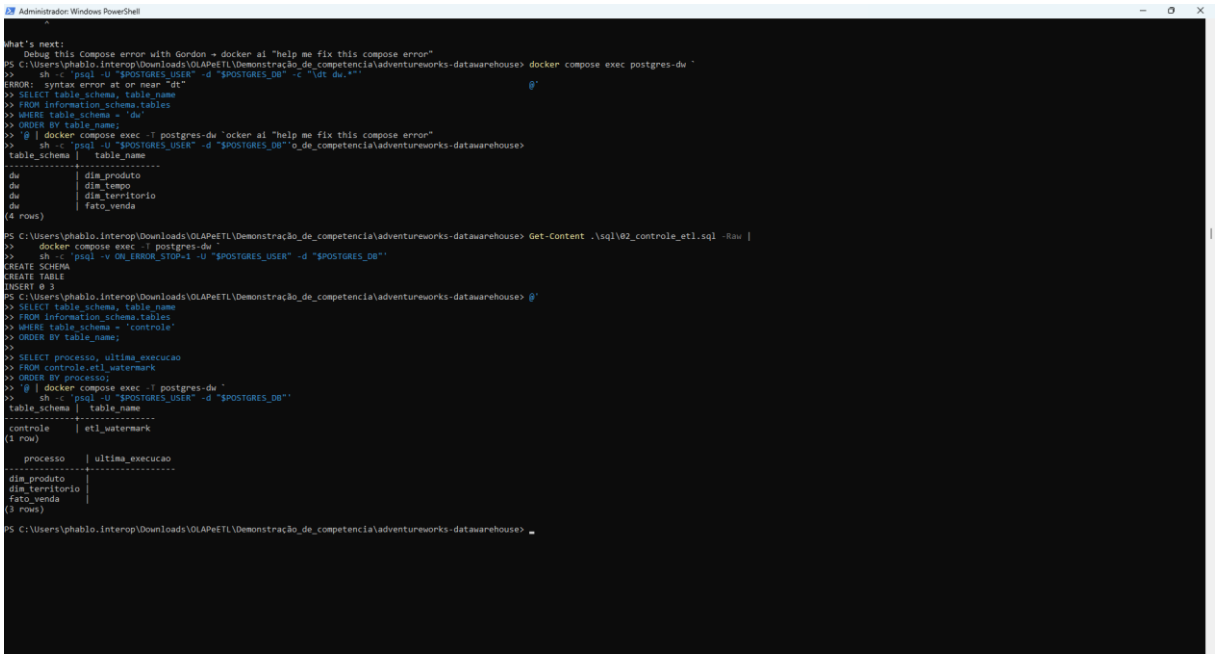


Figura 2 - Evidência da criação das tabelas do DW e da tabela de watermark.

8. Validações técnicas realizadas

Validação	Resultado
Sintaxe dos módulos Python	python -m compileall ./etl executado na imagem apache/airflow:3.3.1 sem SyntaxError.
Bibliotecas	pandas, psycopg2 e pyodbc disponíveis na imagem utilizada.
Driver ODBC	ODBC Driver 18 for SQL Server detectado no container.
Rede Docker → SQL Server	Conectividade a host.docker.internal:1433 validada.
Credenciais SQL Server	Variáveis SQLSERVER_* carregadas pelo --env-file .env; autenticação concluída.
Leitura AdventureWorks	504 produtos e 121.317 detalhes de venda consultados.
Docker Compose	Configuração validada; postgres-dw reconhecido como serviço independente.
PostgreSQL DW	Serviço healthy em 5433; conexão pelo módulo conectar_postgres validada.
Modelo físico	4 tabelas do schema dw criadas e consultadas.
Controle incremental	controle.etl_watermark criado com 3 processos e watermarks NULL.

9. Problemas encontrados e correções aplicadas

Problema	Diagnóstico e correção
SQLSERVER_PASSWORD aparecia como False no container	A linha não existia no .env: o arquivo terminava em SQLSERVER_. A variável completa foi adicionada e o teste passou com todas as SQLSERVER_* = True.
Porta 5432 já alocada ao iniciar dependências do Airflow	O PostgreSQL de metadados tentava publicar 5432 no host. A porta externa foi movida para 5434; internamente permanece postgres:5432.
Comando psql \dt falhou via PowerShell/sh	A barra era alterada no encadeamento de aspas. A validação foi refeita enviando SQL por here-string para stdin do psql.
Nome do projeto Compose herdado do material do professor	Alterado para adventureworks-etl para isolar rede, containers e volumes do projeto final.

10. Segurança e versionamento

O arquivo .env contém credenciais locais e permanece fora do versionamento. O comando git check-ignore .env já havia confirmado que o arquivo está coberto pelo .gitignore. O .env.example deve conter apenas valores de exemplo/placeholders.

Antes do commit, recomenda-se revisar:

```
git status
git diff
git check-ignore .env
```

Arquivos de código, SQL, docker-compose.yml e documentação podem ser versionados. Senhas reais, secrets do Airflow, arquivos .env, logs e backups não devem ser enviados ao repositório.

11. Estado atual e ponto de retomada

Concluído nesta sessão: Consolidação da base ETL, configuração do SQL Server para acesso pelo Docker, criação do PostgreSQL DW, execução do modelo físico, criação do watermark e validação das duas conexões Python.

O próximo teste já definido é a extração isolada de dim_produto, ainda sem gravar no PostgreSQL. O objetivo é confirmar que extrair_dim_produto.py retorna os 504 produtos e as colunas esperadas.

```
docker compose run --rm `
    --no-deps `
    --entrypoint python `
    -e _PIP_ADDITIONAL_REQUIREMENTS= `
    airflow-init `
    -c "from etl.conexoes import conectar_sql_server; from etl.extract.extrair_dim_produto
import extrair_dim_produto; c=conectar_sql_server(); df=extrair_dim_produto(c);
print('Registros extraidos:', len(df)); print('Colunas:', list(df.columns));
print(df.head().to_string(index=False)); c.close()"
```

Após essa validação, a sequência recomendada para a próxima sessão é:

1. Testar transformação de dim_produto.
2. Executar a primeira carga controlada de dim_produto no PostgreSQL.
3. Validar quantidade, chaves e comportamento SCD Tipo 1.
4. Repetir o processo para território e tempo.
5. Testar fato_venda e atualização do watermark.
6. Executar o pipeline incremental completo e uma segunda execução para comprovar ausência de duplicidades.
7. Integrar as rotinas às DAGs do Airflow e registrar evidências para o artigo.

12. Conclusão da etapa

A etapa deixou o projeto em condição segura para iniciar os testes funcionais da ETL. A origem SQL Server e o destino PostgreSQL foram acessados pelo mesmo ambiente de execução que será utilizado pelo Airflow; o modelo dimensional e o mecanismo de watermark existem fisicamente no banco; e a lógica Python está organizada conforme as responsabilidades de extração, transformação, carga e controle incremental. **O próximo marco é validar a movimentação real dos dados, começando pela dimensão produto e avançando progressivamente até a fato e a orquestração completa.**

Observação: este relatório registra o estado efetivamente alcançado até o encerramento da sessão de 23/10/2026. Não foram registradas credenciais reais nem foi considerada concluída qualquer carga que ainda não tenha sido executada e validada.